Programming 3 CW2 Solution Evaluation

Xavier Livingstone-Perez

Student ID: s2339126

Functional Principles

There are 7 principles of Functional Programming:

- Functions are first-class citizens
    - Functions can be assigned to variables and can be passed as arguments to other functions and can be returned from functions
    - I designed the split player data function which is passed as an argument to the scala ".map" higher-order function.

```
// create player object
lines.map(splitPlayerData).map{
case (name, scores) => players = players :+ new Player(name, scores)
}
```

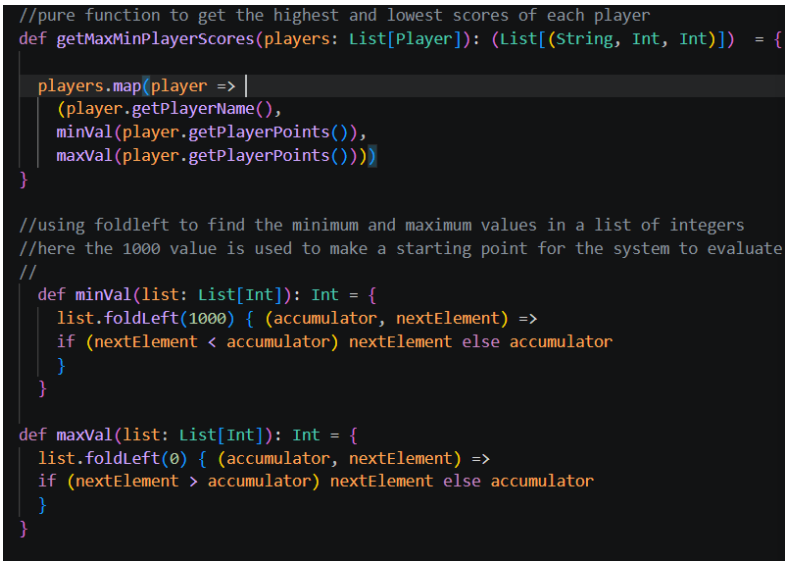
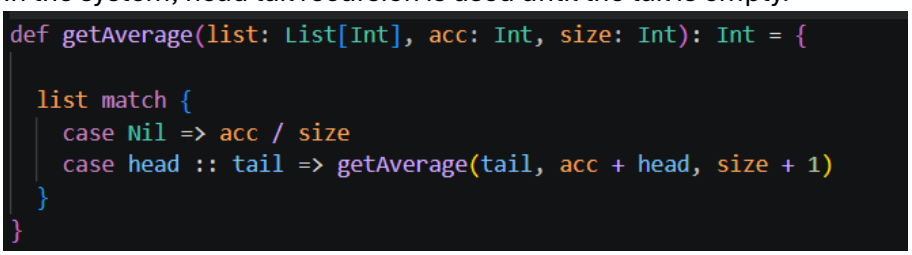
- Functions are deterministic
    - Pure and Impure Functions have also been used
        - Pure functions are deterministic functions which use exact/absolute values within its system and will consistently return the same value when given the same input value
        - The function totalPoints, when given the same input will always return the same output

```
def totalPoints(list: List[Int]): Int = {
  list.reduceLeft { (accumulator, nextElement) =>
  accumulator + nextElement
}
}
```

        - Impure functions are functions which conducts its operations by using external variable values. This means the result of the function on one loop of a system would be different to that of the next loop.
- Functions do not create side effects.
    - Functions are kept pure and deterministic to negate side effects within the system due to impure functions. I adopted this principle through all of my analysis functions
- Data in functional programming is immutable.
    - Functions are able to be made immutable within scala to allow for lists to remain unchanged and is denoted with "val" as opposed to a mutable, changeable list, denoted with "var"

```
def totalPointsOver500(players: List[Player]): List[Player] = {
  players.filter(player => totalPoints(player.getPlayerPoints()) > 500)
}

def totalPoints(list: List[Int]): Int = {
  list.reduceLeft { (accumulator, nextElement) =>
  accumulator + nextElement
  }
}

def displayTotalPointsOver500(players: List[Player])= {

  val playersOver500 = totalPointsOver500(players)

playersOver500.map { player =>
    println("--------------Part 3--------------")
    println(s"${player.getPlayerName()} has total points over 500 with a to
  }
}
```

    This allows for the playersOver500 list to be a new list created for this sole purpose and will be unable to be changed as it has been defined as "val playersOver500"

- Functional programming is declarative.
    - Declarative functions are functions in which the steps to complete a task are described via the functions called rather than using absolute values.
    - Code using declarative functions are hugely reusable and return no side effects
    - 
```
def totalPoints(list: List[Int]): Int = {
  list.reduceLeft { (accumulator, nextElement) =>
  accumulator + nextElement
  }
}
```
- Functional programming uses function composition.
    - Function composition is combining simpler functions within one larger function to gain more scope as to what a single function can carry out.
    - A composite function will call upon many smaller functions within itself to return the final function result.
    - 
```
//pure function to get the highest and lowest scores of each player
def getMaxMinPlayerScores(players: List[Player]): (List[(String, Int, Int)])  = {

  players.map(player =>
    (player.getPlayerName(),
    minVal(player.getPlayerPoints()),
    maxVal(player.getPlayerPoints())))
}

//using foldleft to find the minimum and maximum values in a list of integers
//here the 1000 value is used to make a starting point for the system to evaluate
//
  def minVal(list: List[Int]): Int = {
    list.foldLeft(1000) { (accumulator, nextElement) =>
    if (nextElement < accumulator) nextElement else accumulator
    }
  }

def maxVal(list: List[Int]): Int = {
  list.foldLeft(0) { (accumulator, nextElement) =>
  if (nextElement > accumulator) nextElement else accumulator
  }
}
```
    - The function above, “getMaxMinPlayerScores” calls upon the “.map” function and chains together the minVal and maxVal functions to carry out more tasks within the function
- Functional programming prefers recursion over loops.
    - The system uses recursion in every case in which loops could be used due to the functionality and the ability for a function to recursively call itself until it meets an end condition
    - In the system, head tail recursion is used until the tail is empty.
    - 
```
def getAverage(list: List[Int], acc: Int, size: Int): Int = {

  list match {
    case Nil => acc / size
    case head :: tail => getAverage(tail, acc + head, size + 1)
  }
}
```
    A list is received and the function recursively calls itself until the end of the tail is reached and there is no longer any value in the tail, meaning there are no more elements to be recursively searched through.

Functional Programming Style

The following built in functions below are used here in place of writing out more complex and imperative messy code.

- Map (getRecentWeek)
  - Here ".map" was used in place of looping through a system using classic imperative function
  - The use of ".map" allows for a simple, fast and accurate recursive type operation and will move through the list given rather than a for loop in which every line would need to be defined and can contain side effects

```
// Pure function to get the recent week points of each player
def getRecentWeek(players: List[Player]): (List[(String, Int)]) = {

 //val recentWeek = players.map(_.getPlayerPoints().last) - commented out as is a procedural met

 players.map(player => (player.getPlayerName(), getRecentWeekPoints(player.getPlayerPoints())))

}
```

- Filter (totalPointsOver500)
  - Filter is used so that every item in the list is filtered to the specifications given in the code below using the predicate (player => ___ > 500)

```
def totalPointsOver500(players: List[Player]): List[Player] = {
  players.filter(player => totalPoints(player.getPlayerPoints()) > 500)
}
```

  - Filter is used to alleviate the need to write unnecessary and cluttered lines of code for a system requiring a filter. Imperative code would need a value to be stored, it would need a for loop etc.
- Find and Match/Case (monthAveragePoints)
  - Find and match are used here to verify a user input being equal to a value in a list. It bypasses the need to use custom loops and if else branches to match the user result to the actual result

```
def monthAveragePoints(players: List[Player], player1Name: String, player2Name: String, month: Int): (Int, Int)

 val player1Option = players.find(_.getPlayerName() == player1Name)
 val player2Option = players.find(_.getPlayerName() == player2Name)

 (player1Option, player2Option) match {
   case (Some(player1), Some(player2)) =>
     val player1MonthPoints = player1.getPlayerPoints().slice((month - 1) * 4, month * 4)
     val player2MonthPoints = player2.getPlayerPoints().slice((month - 1) * 4, month * 4)

     //val player1Average = if (player1MonthPoints.nonEmpty) totalPoints(player1MonthPoints) / player1MonthPoin
     val player1Average = if (player1MonthPoints.nonEmpty) getAverage(player1MonthPoints, 0, 0) else 0
     //val player2Average = if (player2MonthPoints.nonEmpty) totalPoints(player2MonthPoints) / player2MonthPoin
     val player2Average = if (player2MonthPoints.nonEmpty) getAverage(player2MonthPoints, 0, 0) else 0

     (player1Average, player2Average)

 }
}
```

- Recursion (getRecentWeekPoints, getAverage)
  - Recursion is used to continuously call itself and utilises a head and a tail element to break up a list.

```
def getAverage(list: List[Int], acc: Int, size: Int): Int = {

  list match {
    case Nil => acc / size
    case head :: tail => getAverage(tail, acc + head, size + 1)
  }
}
```

- The getAverage function takes in a list of integers, an accumulator initialised to 0, and a size of the list
- The list will, for each player in it, loop through the weekly points, add them all up and then divide it by how many items were in the list to get the average points
- An imperative function for this would be more complex and time consuming

- FoldLeft (minVal maxVal)
  - The foldLeft function, in this case is used to recursively loop through a system and define the lowest value in a list. It is initialised at 1000 as to allow for every number to be guaranteed a smaller value.
  - Fold left is used instead of reduceLeft as foldLeft requires an initial value to be input, whereas reduceLeft will take the first value of the collection as the initial value. This doesn't work for a function which is searching for a minimum or a maximum value as the data may not be in order

```
def minVal(list: List[Int]): Int = {
  list.foldLeft(1000) { (accumulator, nextElement) =>
  if (nextElement < accumulator) nextElement else accumulator
  }
}
```

- ReduceLeft (totalPoints)
  - Higher order functions are a type of function that is generally built in to the scala framework ".reduceLeft", which recursively loops through the system from the tail side (left) and works its way to the head allowing for a faster and simpler process

```
def totalPoints(list: List[Int]): Int = {
  list.reduceLeft { (accumulator, nextElement) =>
  accumulator + nextElement
  }
}
```

  - Using imperative functions would require significantly more code to utilise and would make use of while loops and immutable variables which can be bypassed using the reduceLeft function

Comparison of Functional and Imperative Style

- Functional programming can be hard to think about initially, but recursion, once you can think of problems that way, makes it easier to have functions that have less side effects and less messy code and generally smaller functions with less complexity.
- The code below shows the difference between using functional vs imperative programming techniques

```
def maxVal(list: List[Int]): Int = {
  list.foldLeft(0) { (accumulator, nextElement) =>
  if (nextElement > accumulator) nextElement else accumulator
  }
}

def maxValImperative(list: List[Int]): Int = {
  var max = 0
  for (num <- list) {
    if (num > max) {
      max = num
    }
  }
  max
}
```

  It can be deduced that the maxValImperative function is more complex and therefore more time consuming to write.
- If given free choice of language other than scala, I would try to make use of functional programming techniques such as avoiding the use of global variables within functions and making my functions more pure, making testing easier. This can be adapted in Java, Python or any other coding language.